

Generalized Dynamical Systems as a Substrate for Complex Cryptoeconomic Modeling

1. Central Thesis & Irreducible First Principles

The central thesis of the provided research substrate asserts that traditional mathematical modeling paradigms—specifically those relying exclusively on continuous-time ordinary differential equations mapped over real-valued Euclidean spaces—are fundamentally insufficient for designing, simulating, and verifying modern socio-technical and cryptoeconomic systems. To bridge the widening gap between formal mathematical verification and computer-aided software implementation, the substrate proposes a return to and expansion of Generalized Dynamical Systems (GDS). Rooted deeply in the pioneering work of Emilio Roxin and others in the 1960s concerning differential inclusions and contingent derivatives, GDS provides a mathematically rigorous environment where the state space is not artificially restricted to smooth manifolds. Instead, it accommodates arbitrary data structures, allowing categorical sets, dynamic network topologies, and relational databases to be mapped to themselves while remaining closed under composition. The deepest common epistemological idea uniting this framework is the strict mathematical decoupling of deterministic system mechanisms—the immutable laws of motion or protocol code—from non-deterministic environmental and behavioral inputs, such as the choices made by autonomous agents. By formalizing the boundaries of what is mathematically possible (reachability) rather than attempting the fraught task of predicting exactly what will happen (behavioral forecasting), GDS enables the robust engineering of computer-aided governance and enforceable algorithmic contracts.

This generalized framework rests upon several irreducible first principles that function as the axiomatic foundation for all subsequent modeling, simulation, and cryptoeconomic analysis. The first and most foundational principle is the concept of State. In classical physics, state implies numerical vectors of position and momentum. In GDS, the state is the irreducible snapshot of the system, defined as a minimal sufficient statistic of the system's entire history at a specific temporal index, such that no further historical data is required to compute the subsequent transition. This state is decoupled from strictly numerical arrays; it can natively represent a complex, expanding data record, such as the ever-growing Unspent Transaction Output (UTXO) set in the Bitcoin network, or a dynamic graph of token holders.

Operating upon this state is the Mechanism, which represents the deterministic, procedural automation or the "laws of motion" that transition the system from one state to the next. The mechanism is entirely conditional on the current state and a specific input, embodying the designed infrastructure, such as the compiled bytecode of an Automated Market Maker (AMM) or a smart contract. However, because the system interfaces with an unpredictable external environment, it must define Constraints through Admissibility. Admissibility acts as the bounding box of reality. Since inputs from human agents or external oracles are inherently non-deterministic, the system must establish an Admissible Input Space that rigorously delineates the boundaries of what actions are physically, logically, or cryptographically legal

given the current state.

Within these constraints exists Behavior, which is explicitly conceptualized as the specific selection of an action from the admissible set. Behavior is treated as an external, often unobservable, or highly uncertain stochastic process. GDS philosophically isolates behavior from the mechanism, formalizing the reality that while a system designer enjoys absolute control over the mechanism's code, they can only make game-theoretic or statistical assumptions about human behavior. The synthesis of these elements manifests as Dynamics, or the transition map over time, encapsulating how the system evolves when an agent applies a behavioral policy within the immutable boundaries of the mechanism.

Time itself is reconsidered as a flexible first principle. Rather than being rigidly bound to continuous flows (dt) or standardized discrete ticks, time in GDS is generalized as an index of evolution. It can represent asynchronous event-sequence time, such as block arrivals on a decentralized ledger or transactional events popping into existence via a Poisson process, allowing the temporal dimension to morph according to the specific phenomenon being modeled. Finally, the principle of Observation serves as the epistemic window into the system. Acknowledging that the full, high-dimensional state or the true behavioral mapping of agents is rarely perfectly knowable to external analysts, the system relies on carefully defined metrics, key performance indicators (KPIs), or sensors to extract partial information, which subsequently drives parameter selection and control feedback loops.

2. Intellectual Tradition & Comparative Framework Analysis

The Generalized Dynamical Systems framework detailed in the substrate emerges from a highly interdisciplinary synthesis, anchoring itself primarily in advanced non-linear control theory while extending aggressively into the domains of Model-Based Systems Engineering (MBSE) and institutional economics. Its pure mathematical origins lie in the study of differential inclusions and contingent equations, pioneered by Emilio Roxin in 1965 to handle control systems encumbered by bounded uncertainties and discontinuous right-hand sides. However, the substrate radically repurposes these mid-century mathematical tools for the digital age, shifting the focus from mechanical systems and aerospace trajectories to complex adaptive socio-technical systems and tokenized economies.

To properly contextualize the epistemological novelty and utility of GDS, it is necessary to compare it critically with other prominent modeling traditions, highlighting both its complementary strengths and its deliberate departures from orthodox methodologies.

Intellectual Tradition	Core Methodology & Assumptions	Comparison with Generalized Dynamical Systems (GDS)
Classical Control Theory	Relies on ordinary differential equations (ODEs) mapped over real-valued Euclidean spaces ($\mathbb{R}^n$). Seeks global asymptotic stability via Lyapunov functions for well-understood, continuous physical plants.	GDS inherits the architecture of control theory (plant, controller, observer) but abandons the strict requirement for smooth, real-valued manifolds. Where classical control seeks global stability in continuous spaces, GDS seeks reachability bounding and invariant

Intellectual Tradition	Core Methodology & Assumptions	Comparison with Generalized Dynamical Systems (GDS)
		preservation across discrete, structured topological spaces (e.g., dynamic network graphs).
Complex Systems & Agent-Based Modeling (ABM)	Focuses on emergent macro-behaviors arising from the heuristic micro-interactions of heterogeneous, interacting agents. Highly expressive but computationally challenging to verify formally.	GDS effectively inverts the ABM paradigm. Instead of focusing on generating emergent behavior through agent simulation, GDS maps the rigid structural boundaries (the mechanism) within which behavior occurs. ABMs forecast <i>what will happen</i> , whereas GDS maps <i>what can happen</i> regardless of specific agent heuristics.
Differential Dynamic Logic (dL) & Hybrid Systems	A deductive verification calculus (championed by André Platzer) for hybrid systems mixing discrete jumps and continuous differential equations (e.g., automated train controls).	Both dL and GDS focus heavily on invariant preservation and safety verification. However, dL is primarily a formal logic for automated theorem proving relying on first-order real arithmetic. GDS, particularly via tools like cadCAD, functions as a computational ontology prioritizing simulation-driven software engineering and algorithmic policy over purely deductive proofs.
Model-Based Systems Engineering (MBSE) & INCOSE	Seeks to replace static, document-centric engineering workflows with formal, computable models to manage system lifecycles.	GDS serves as the explicit mathematical formalism that provides MBSE with a rigorous computational engine. It translates abstract business requirements into mathematical blueprints, specifically enabling MBSE to handle socio-technical systems containing algorithmic policies coupled with highly unpredictable human agency.
Statistical Learning & Data-Driven Science	Seeks to infer system dynamics and governing equations entirely from empirical data, utilizing neural networks and deep learning.	GDS remains fundamentally mechanistic and structural. It insists that the smart contract or protocol layer is known with absolute deterministic certainty. Machine learning may be used

Intellectual Tradition	Core Methodology & Assumptions	Comparison with Generalized Dynamical Systems (GDS)
		within GDS to infer the hidden behavioral policy or to classify fraud, but the core mechanism remains a strict, defined mathematical object, preventing "black box" causality loss.

The comparison with Differential Dynamic Logic (dL) and hybrid systems is particularly illuminating. In the domain of cyber-physical systems, such as the European Train Control System, Platzer's dL successfully utilizes Skolemisation and real-valued free variables to prove absolute collision avoidance properties by evaluating differential constraints algebraically without needing to solve them outright. The continuous evolution of the train's position and the discrete braking decisions are modeled with rigorous modalities. However, cryptoeconomic systems present a different class of problem. While a train operates in a predictable Euclidean geometry, decentralized financial systems operate in abstract, categorical spaces where state dimensionality can instantly expand (e.g., the instantiation of a new liquidity pool or the generation of infinite wallet addresses). The GDS substrate argues that while the deductive rigor of dL is aspirational, the sheer structural dynamism of cryptoeconomics necessitates a shift toward the contingent derivative approach and differential inclusions, where discrete simulations and parameter sweeps can practically approximate safety bounds in intractable topological spaces.

Furthermore, GDS interfaces uniquely with modern statistical learning. As noted in the substrate's discussions on fraud detection in Gitcoin grants, machine learning algorithms are utilized not to replace the system's structural model, but to operate within the defined Admissible Action Set. The inference engine classifies whether a realized, reachable state was generated by honest behavior or fraudulent collusion by inverting the state transition mapping to evaluate the likelihood of specific actions having been selected. This maintains the integrity of the causal model while leveraging modern data science to parameterize the unknown variables of human behavior.

3. Core Concepts & Mathematical Language

The substrate relies on a sophisticated and highly specific mathematical language designed to generalize classical dynamical concepts so they can natively support arbitrary, non-numeric data structures. The interplay between these mathematical tools provides the operational machinery of GDS, allowing engineers to transition from abstract theory to computable code. A foundational concept in this language is Reachability and Attainability. In classical systems, a specific input generates a specific, deterministic trajectory. In GDS, because the exact behavioral input is drawn non-deterministically from an allowed set, the system's future is not a single line but a cone of possibilities. Reachability defines the exact topological subset of the total state space that a system can physically evolve into, starting from an initial state and subjected to all conceivable sequences of admissible actions. This concept is paramount because establishing safety in cryptoeconomic systems is synonymous with proving that catastrophic states—such as a drained liquidity pool, an under-collateralized lending protocol, or an infinite minting loop—are mathematically unreachable from the valid initialization states. This reachability is governed strictly by Admissibility. Admissibility constitutes the strict

mathematical bounding of the input space. An action u is admissible if and only if it belongs to the set U_x , which represents the physical, logical, or cryptographic constraints of the system given the current state x at time t . This serves as the primary lever for mechanism design: by deliberately restricting the boundaries of admissibility (e.g., placing limits on transaction sizes or requiring cryptographic signatures), the protocol designer effectively strangles the reachability tube, excising dangerous state spaces from the realm of the possible.

To formalize this mathematically, GDS employs Differential Inclusions. A generalization of differential equations, a differential inclusion dictates that the rate of change of a state is not a single deterministic value, but belongs to a predefined set. Expressed as $\dot{x} \in F(x)$, this mathematical formulation allows the system to model environmental uncertainty natively. When the state space is highly abstract and does not permit standard differentiable limits, GDS utilizes Contingent Derivatives. The contingent derivative evaluates the geometry of the reachable set as the system steps forward in time. This approach enables the rigorous study of system boundaries without demanding smooth, continuous functions.

The mathematical machinery relies heavily on proving that the distance between the attainability correspondence (the set of all possible futures) and the mandated constraint set remains within a tight, acceptable error bound ϵ . Because Euclidean distance is meaningless in categorical data structures, GDS utilizes the Hausdorff distance (d_H). The validation condition is expressed mathematically as ensuring that for all (x, t) , $d_H(C(x, t; g), C(x_0, t_0; g)) < \epsilon$, where C represents the constraint set parameterized by the behavioral map g .

Central to verifying these conditions is the concept of Invariants or Conserved Properties. Invariants are symmetries or mathematical truths that remain absolutely constant regardless of the state transitions occurring within the system. In decentralized finance, the most recognizable invariant is the constant product formula ($x \cdot y = k$) utilized by Automated Market Makers. Proving that a discrete mechanism flawlessly preserves an underlying continuous invariant across all admissible inputs is the central task of validating a GDS model.

When mapping this continuous, abstract topology to executable software architectures, the system utilizes Block Diagrams and compositional morphisms, drawing theoretical inspiration from Applied Category Theory. The state transition map, elegantly summarized as $x_{t+1} = h(x_t) \equiv f(x_t, g(x_t))$, becomes a tensor-like operation in computational frameworks like `cadCAD`. Complex data payloads (state spaces) are passed through blocks (mechanisms) governed by strict routing logic and type-safety definitions, transforming the pure mathematics of contingent equations into discrete, verifiable software simulations.

4. Systems View

The research substrate projects a highly specific, rigorous ontology regarding what constitutes a "system." It decisively rejects the naive, colloquial view of a system as a mere collection of interacting parts. Instead, it defines a system computationally, topologically, and epistemologically through its defined boundaries, interfaces, and state-transitions. This worldview demands strict categorical boundaries between what is designed, what is assumed, and what is observed.

In this paradigm, a State is fundamentally an information container. It is the minimal, sufficient data structure required to completely capture the condition of the system at a given temporal index, effectively summarizing all relevant historical path-dependencies such that no further historical data is required to compute the next algorithmic transition. In this expansive view, a traditional relational database, a dynamically evolving graph of interacting agents, or a

cryptographic ledger of token balances are all equally valid state representations.

A Mechanism represents the immutable, deterministic mapping of the system. It is the set of rules—the inescapable physics of the modeled environment or the compiled bytecode of a smart contract—that dictates exactly how the state will mutate given a specific, admissible input. Mechanisms are the designed artifacts of the system, the elements over which the engineer exercises total authorial control.

Conversely, Behavior is conceptualized as the non-deterministic engine propelling the system through its state space. It is the hidden policy, the internal decision-making process of an agent, or the random noise of nature that selects a specific input u from the admissible set U_x . The defining philosophical stance of GDS is that the system designer does not, and cannot, control behavior. The designer can only make statistical or game-theoretic assumptions about it, or strictly bound its impact via admissibility rules.

This distinction is heavily emphasized in the substrate's recurring pedagogical example of the crosswalk. In this model, the physical street, the traffic laws, and the painted crosswalk constitute the Mechanism and define the Admissibility sets. The decision of a pedestrian to utilize the crosswalk, or to dangerously jaywalk to reach a destination faster, constitutes Behavior. The reachability analysis seeks to understand the boundaries of traffic flow regardless of pedestrian rationality, while the mechanism design seeks to place the crosswalk (the parameter) in a location that maximizes safety metrics based on anticipated behavioral distributions.

A Model, therefore, is not a perfect digital replica of reality, but a dimensionally reduced representation constructed for a specific analytic view or purpose. A rigorous model explicitly acknowledges its own assumptions, particularly its behavioral assumptions, and defines its boundaries. It is an abstraction designed to render computational verification or simulation computationally tractable.

Validation within this worldview is the rigorous process of proving—either analytically through mathematical reachability analysis or numerically through extensive Monte Carlo simulation parameter sweeps—that the system's mechanisms, when subjected to the full spectrum of admissible behaviors, will invariably preserve the predefined invariants and never breach the threshold of a catastrophic state.

Finally, Explanation in the GDS framework is entirely structural and causal. To explain a system phenomenon is not merely to narrate a sequence of events. Rather, it is to map a realized trajectory of states backward through the inverse of the mechanism (f^{-1}) to identify the specific subset of admissible actions that mathematically could have generated the observed outcome. Explanation in GDS serves to surgically separate structural inevitabilities—the faults of the mechanism—from behavioral choices, allowing engineers to pinpoint exactly where an adversarial exploit manipulated the bounds of admissibility.

5. Research Themes & Concept Map

The material encompassing the Generalized Dynamical Systems substrate can be synthesized into five major research themes, which collectively structure the ongoing development and practical application of the paradigm.

Theme Name	Main Idea	Representative Concepts	Key Scientific Question
Formal Abstraction & MBSE	The critical process of translating qualitative,	System Boundaries, Conceptual Blueprints,	How can human design goals be losslessly

Theme Name	Main Idea	Representative Concepts	Key Scientific Question
	ambiguous business requirements into rigorous mathematical specifications before a single line of production code is written.	Mathematical Specification, Contextual Interfaces.	translated into mathematically verifiable system constraints?
Topological State Dynamics	Modeling systems where the dimensionality of the state space itself expands, contracts, or mutates over time, breaking classical metric space assumptions.	Arbitrary Data Structures, Expanding State Spaces (e.g., UTXO sets), Morphisms.	How do we mathematically define stability and convergence when the topological space is dynamically changing?
Reachability & Safety Verification	Defining the absolute theoretical boundaries of system evolution rather than attempting to predict specific, highly uncertain behavioral outcomes.	Differential Inclusions, Contingent Derivatives, Attainability Correspondence, Hausdorff Metrics.	Does there exist any mathematically admissible action sequence that can force the system into a failure state?
Behavioral Isolation & Admissibility	The strict epistemological separation of the deterministic technological infrastructure from the non-deterministic human actors interacting with it.	Admissible Action Sets, Behavioral Policies, Plant vs. Controller.	How can precise mechanism design securely bound the impact of adversarial, irrational, or highly coordinated agent behavior?
Simulation & Computational Implementation	Moving from continuous mathematical proofs to discrete, iterative computational models using tools like cadCAD to achieve practical engineering goals.	Discretization, Monte Carlo Parameter Sweeps, Metric KPIs, Block-Diagram Morphisms.	How can discrete computational event-sequence simulations accurately approximate and verify the properties of continuous mathematical invariants?

Concept Map (Hierarchy of Dependencies)

The architecture of Generalized Dynamical Systems operates through a strict hierarchy of dependencies, flowing from abstract philosophy to executable code.

At the apex is the **Metaphysical Foundation**, which establishes the fundamental

epistemological separation between the deterministic technological infrastructure (the Plant/Mechanism) and non-deterministic agency (Behavior). This foundational stance dictates that a system can only be modeled if its boundaries and external interfaces are explicitly defined, preventing the conflation of rules and choices.

Depending directly upon this metaphysical separation is the **Mathematical Formalism** of GDS itself. To map a system where inputs are non-deterministic, the framework relies on the definitions of State Spaces (X), Admissible Sets (U_x), and State Transition Maps (h). This formal layer is mathematically enabled by mid-century tools repurposed for discrete data—specifically Contingent Derivatives, Differential Inclusions, and Hausdorff Metrics, which allow for the measurement of boundaries in abstract spaces.

This mathematics is then operationalized into **Systems Engineering (MBSE)**. The abstract mathematical boundaries are used to execute Mechanism Design and Blueprinting. Engineering teams translate qualitative client goals into formal Invariants and Constraints. This stage relies wholly on the mathematical Reachability Analysis and Local Controllability established in the previous layer to prove that the designed mechanism theoretically restricts the state to safe zones.

Finally, the theoretical engineering blueprint is instantiated into **Computational Implementation**, manifesting as a Digital Twin or complex simulation environment (e.g., cadCAD). This requires discretizing the continuous GDS mathematics, organizing state transition maps into Block-Diagram Morphisms. This executable layer is enabled by Monte Carlo Parameter Sweeps and Metric KPIs, which test the theoretical blueprint against simulated computational reality to validate the initial business requirements.

6. Conceptual & Methodological Gap Analysis

While the GDS framework presents a highly robust, mathematically rigorous approach to systems modeling, a critical evaluation of the research substrate reveals several underrepresented topics, conceptual gaps, and missing mathematical tools that currently limit its immediate, frictionless applicability to certain classes of real-world complex systems.

Endogenous State-Space Expansion & Dimensional Explosion: The framework theoretically supports state spaces that change in dimensionality over time. A cited example is the Bitcoin network, where new wallet addresses are continuously generated, expanding the state vector infinitely. However, the mathematical machinery provided heavily relies on differential inclusions and contingent derivatives that, historically, assume a fixed—albeit abstract—topological space. When the dimensionality N of the system grows unboundedly as a function of time or adversarial action (e.g., a Sybil attack generating millions of dummy accounts), calculating the reachability tube or applying the Hausdorff metric becomes computationally intractable. The substrate lacks the specific mathematical tools required to perform dimension-agnostic invariant verification at scale without succumbing to the curse of dimensionality.

Endogenous Role Selection and Hidden Types: In the substrate's extensive discussion of the pedagogical crosswalk example, a critical limitation of the framework is exposed: GDS struggles significantly when actors endogenously change their type or role mid-simulation. GDS relies structurally on the clean, mathematical separation of the mechanism (f) and the behavioral policy (g). However, if an agent's policy is state-dependent such that it alters the fundamental topology of the system's interactions—for example, a pedestrian deciding to get into a car and become a driver, or a decentralized exchange liquidity provider instantly shifting their capital to become an aggressive arbitrageur—the separation between the plant and the controller

degrades. The current framework lacks a robust mechanism for modeling these dynamically coupled, multi-role agent typologies without forcing a massive, unwieldy dimensional lift that breaks computational feasibility.

Compositional Verification of Nested Systems: While the substrate discusses "systems of systems" and relies on block diagrams for chaining distinct mechanisms together, the formal mathematical tools for compositional verification remain underrepresented. If Subsystem A (e.g., a lending protocol) is proven mathematically to be locally controllable, and Subsystem B (e.g., an automated market maker) is also proven locally controllable, the framework currently lacks the applied category theory or compositional game theory machinery to guarantee analytically that the union of A and B remains stable. The assumption that stable components yield a stable whole is a known fallacy in complex systems, and currently, GDS relies on exhaustive global simulation rather than compositional deductive proofs to bridge this gap.

Continuous-to-Discrete Epistemic Friction: A significant methodological gap exists in the translation layer between theoretical mathematics and computational execution. The substrate heavily leverages continuous-time mathematics (differential inclusions, contingent derivatives) to theoretically prove bounding properties and invariant preservation. However, it relies on discrete-time event updates within the cadCAD framework for actual software implementation. The methodological gap lies in formally proving that the discrete event-sequence simulation perfectly bounds the continuous mathematical reality without introducing numerical integration errors, floating-point rounding artifacts, or discretization gaps that could mask deep systemic vulnerabilities in cryptoeconomic protocols.

Out-of-Band Attack Modeling and Time Manipulation: The framework models behavior as a draw from an admissible set at a given time t . However, in real-world cryptoeconomic systems, the fundamental assumption of sequential time is often weaponized. The framework does not natively address mempool dynamics—specifically Miner Extractable Value (MEV) and front-running. Adversaries do not merely select actions; they manipulate the chronological ordering of the event-sequence itself, creating "context-breaking" vulnerabilities. Extending the GDS transition map $h(x)$ to account for asynchronous, competitive transaction reordering and the temporal insertion of out-of-band actions (like flash loans sourced from external protocols) remains a vital missing piece of the theoretical puzzle.

7. 10 Original Ranked Research Questions

The following research questions emerge directly from the gaps, implications, and theoretical horizons of the Generalized Dynamical Systems substrate. They are ranked by an aggregate estimation of their theoretical novelty, technical feasibility, and potential scientific impact in the fields of systems engineering and cryptoeconomics.

Rank	Research Question	Novelty	Feasibility	Impact	Justification
1	Compositional Invariant Preservation: How can applied category theory be integrated with GDS	High	High	High	Solves the critical issue of DeFi composability risk. If components are safe, under what exact

Rank	Research Question	Novelty	Feasibility	Impact	Justification
	differential inclusions to formally prove that invariant properties are preserved when multiple isolated AMM mechanisms are composed into a single routing network?				categorical conditions is the composite system guaranteed safe without requiring infinite simulation?
2	Dynamic Dimensionality Metrics: What novel topological metrics can replace the Hausdorff distance to evaluate the convergence of attainability correspondences in state spaces where the number of dimensions scales as an unpredictable Poisson process?	High	Medium	High	Addresses the fundamental limitation of modeling blockchain UTXO sets or Sybil attacks where traditional distance metrics fail due to dimensional explosion.
3	Stochastic Reachability: How can stochastic bounds be formally introduced into the contingent derivative approach to generate	Medium	High	High	Transforms binary reachability (possible/impossible) into probabilistic risk gradients, making the framework immediately useful for

Rank	Research Question	Novelty	Feasibility	Impact	Justification
	probabilistic reachability tubes, allowing for rigorous Value at Risk (VaR) calculations in algorithmic stablecoins?				traditional financial risk management and institutional capital allocation.
4	Admissibility Discovery via Manifold Learning: Can unsupervised machine learning algorithms be utilized to infer the true empirical boundaries of an Admissible Action Set (U_x) by analyzing historical state trajectories of exploited smart contracts?	Medium	High	High	Bridges data science and GDS. Instead of guessing the bounds of behavior, this allows systems to autonomously learn the effective boundaries of the protocol based on empirical on-chain data.
5	Epistemic Bridging of Time & Discretization: Under what precise mathematical conditions does the event-sequence discretization of a differential inclusion perfectly preserve the safety	Medium	High	Medium	Essential for proving that the cadCAD simulations are not introducing false positives or masking vulnerabilities due to discrete time-stepping artifacts.

Rank	Research Question	Novelty	Feasibility	Impact	Justification
	invariants of the continuous-time idealization in financial state machines?				
6	Endogenous Role Shifting and Hidden Types: How can the GDS state transition map be augmented to natively handle hidden agent types and endogenous role transitions (e.g., pedestrian to driver) without requiring intractable increases in state-space dimensionality?	High	Medium	Medium	directly addresses a highlighted flaw in the pedagogical crosswalk model, expanding the framework's capacity to model fluid human behavior without breaking the plant/controller dichotomy.
7	Out-of-Band Temporal Attack Modeling: How can GDS be mathematically extended to model "context-breaking" vulnerabilities, such as MEV and flash loans, where an agent manipulates the perceived chronological ordering of the	High	Medium	High	MEV is the primary security threat in modern blockchains. If GDS cannot natively model time-reordering in the mempool, it cannot fully verify decentralized exchange security.

Rank	Research Question	Novelty	Feasibility	Impact	Justification
	event sequence itself?				
8	Automated Mechanism Synthesis: Can the GDS mathematical specification be inverted such that, given a set of desired invariants and a bounded admissible action space, the optimal state transition function $f(x)$ can be algorithmically synthesized?	High	Low	High	Pushes GDS from a descriptive and verification tool into an automated generative design tool, fundamentally altering how smart contracts are engineered.
9	Local Controllability in Pure Graphs: What are the necessary and sufficient conditions for establishing local controllability (a basin of attraction) over a GDS where the state is represented exclusively by a directed, weighted, and dynamically shifting graph?	High	Low	Medium	Crucial for designing and steering decentralized autonomous organizations (DAOs) and social networks where state is purely relational rather than numeric.
10	Hamilton-Jacobi on	High	Low	Medium	Adapts classical

Rank	Research Question	Novelty	Feasibility	Impact	Justification
	Categorical States: To what extent can verification functions derived from the Hamilton-Jacobi equation be adapted for non-metric, categorical state spaces common in governance voting mechanisms?				optimal control theory for digital democracy, attempting to mathematically solve for optimal governance trajectories.

8. Methodological Patterns & Staged Research Agenda

A deep reading of the notebook reveals that it implicitly teaches a rigorous, non-linear research method that heavily prioritizes structural safety and verifiable bounds over the traditional economic goal of behavioral prediction. This methodology can be defined as **Structural Isolation and Simulation-Driven Calibration**.

The pattern operates in iterative, rigorous loops. First, the researcher engages in Mechanistic Modeling, defining the system boundaries to physically, logically, or cryptographically isolate the deterministic laws of motion from the external environment. Second, the researcher performs Structural Analysis via reachability mathematics to prove the theoretical bounds of the system, answering the question: what is the absolute worst-case scenario if the admissible set is fully exploited by adversarial actors? Third, the theoretical model is discretized into a computational simulation environment like cadCAD. Here, the researcher conducts Simulation and Calibration, utilizing Monte Carlo methods to sweep parameters against stochastic behavioral models. This calibrates the mechanisms so that the system is locally controllable toward desired KPIs, despite behavioral uncertainty. Finally, the loop closes with Empirical Grounding: the system is deployed, actual state trajectories are observed, and the true behavioral policy $g(x)$ is inferred, which feeds back to tighten the admissible set or update the core mechanism in a subsequent version.

If this rich substrate were to form the foundation of a staged PhD or institutional research agenda, it would follow a highly structured, phased trajectory moving from pure mathematics to applied cryptoeconomic engineering.

The **Foundations phase (Years 1-2)** would concentrate entirely on formalizing the ontology of state spaces, inputs, and state transition maps for discrete digital networks. This involves establishing the foundational mathematical proofs bridging Roxin's mid-century contingent derivatives and differential inclusions with modern discrete-event software systems. The primary

output would be the development of robust MBSE block diagrams for canonical, isolated cryptoeconomic mechanisms, such as simple Automated Market Makers or basic bonding curves.

The **Intermediate phase (Years 2-3)** would shift focus to implementation and computational simulation. The goal would be implementing GDS mathematically in cadCAD, ensuring strict theoretical type-safety between domain and codomain mappings across complex tensor operations. Researchers would develop standardized frameworks for parameter selection under uncertainty, running massive Monte Carlo sweeps to conduct local controllability analyses on single-mechanism systems, successfully defining robust, provable basins of attraction.

The **Advanced phase (Years 3-4)** would tackle the known limitations and conceptual gaps of the framework. This entails analyzing systems with dynamic, expanding dimensionalities, such as unbounded network graphs or infinitely expanding UTXO sets. It would heavily feature the integration of Reinforcement Learning agents into the simulation environment to relentlessly test mechanism resilience against optimal adversarial behavior. A key theoretical milestone would be developing formal definitions for compositional GDS, proving mathematically that interconnected stable subsystems do not interact to generate emergent systemic instabilities. Finally, the **Capstone problems (Year 5+)** would push the boundaries of systems engineering into automation and real-time defense. Researchers would focus on Automated Mechanism Design—attempting to compile natural language or formal business requirements directly into mathematically verified, ready-to-deploy GDS specifications. Concurrently, efforts would focus on real-time system identification, developing on-chain algorithmic observers that dynamically invert the GDS map to classify anomalous or fraudulent behavior in real-time based on minute deviations in reachable state trajectories.

9. Cryptoeconomic Systems Application & Extensions

To build a rigorous, mature science of cryptoeconomic systems, the Generalized Dynamical Systems framework is highly and uniquely applicable, but it requires specific domain tailoring. Following Voshmgir and Zargham's foundational definitions, cryptoeconomic systems consist of three distinct structural levels: individual autonomous actors at the micro-level, economic policies embedded in software at the meso-institutional level, and emergent properties arising at the macro-level. The GDS framework operates natively and most powerfully at the meso-institutional level, providing the explicit mathematics of policy setting and algorithmic governance.

Directly Applicable Elements: The most direct application of GDS lies in the conceptualization of Smart Contracts as Mechanisms. In blockchain systems, the mechanism (f) is perfectly known without error; it is the compiled, immutable bytecode of the smart contract. GDS is arguably the native mathematical language of smart contracts because the strict separation of deterministic state updates (the code) and external inputs (the transaction payload) is enforced absolutely by the blockchain protocol itself.

Furthermore, Admissible Action Sets map flawlessly to Protocol Rules. Cryptography and consensus rules dictate exactly what transactions are legally valid on-chain. Thus, U_x is strictly definable, making reachability analysis highly precise and uniquely suited for auditing smart contract security. This facilitates Invariant-Based Design. Decentralized Finance (DeFi) primitives rely almost entirely on mathematical invariants, such as the constant product formula in Uniswap. GDS's obsessive focus on invariant preservation maps directly to AMM design, validation, and verification. BlockScience's work on the Commons Stack, utilizing Augmented

Bonding Curves (ABCs) as automated market makers to act as boundaries between external economies and cause-focused token economies, is a premier example of GDS applied to public goods funding.

Necessary Modifications: Despite these strengths, the framework requires modification to handle the specific realities of decentralized networks. Crucially, GDS must be adapted to account for Value Capture and Token Policy. While GDS easily models token flow and state transitions, the standard formulation must be modified to incorporate the endogenous factors that shape a token's actual market value. The state must be expanded to represent not just numeric token balances, but the relative purchasing power or off-chain real-world asset backing of the token, which introduces highly complex, volatile exogenous variables into the admissibility constraints.

Additionally, the treatment of time requires significant revision to address Mempool Dynamics and Front-Running. In the idealized GDS substrate, time is treated cleanly and sequentially. In actual cryptoeconomics, the ordering of events within a discrete block (the mempool) allows for adversarial action, namely Miner Extractable Value (MEV). The GDS state transition map $h(x)$ must be fundamentally modified to account for asynchronous, highly competitive transaction reordering, where the sequence of admissibility is itself a manipulatable variable.

Required New Theories: To fully mature, cryptoeconomics operating via GDS requires the development of entirely new theoretical branches. First, a theory of Adversarial Network Topology is needed. GDS must bridge with advanced network theory to mathematically model Sybil attacks, where an adversarial agent artificially inflates the dimensionality of the state space by spawning thousands of addresses to manipulate governance mechanics and voting weights. Second, the field requires a robust theory of Cryptoeconomic Controllability. Classical control theory assumes a unified, omnipotent controller actively steering the plant. Cryptoeconomic systems, by definition, lack centralized control. Therefore, the framework requires a theory of *decentralized controllability*, where the system is steered not by a single entity manipulating inputs, but by designing complex incentive gradients that probabilistically guide a decentralized, uncoordinated swarm of self-interested actors toward the desired institutional design goals.

10. Final Synthesis

The research notebook on Generalized Dynamical Systems articulates a profound, paradigm-shifting methodology for the engineering of complex socio-technical systems. It decisively abandons the historical hubris of behavioral prediction—the flawed economic idea that we can definitively model or control exactly what humans will do—and replaces it with the mathematical rigor of structural bounding.

By systematically lifting the strict deterministic logic of classical control theory out of the continuous geometry of physical space and deploying it into the abstract, discrete topological data structures of software networks, GDS provides an indispensable mathematical chassis for the digital age. It asserts a highly specific, rigorous worldview: the engineer's primary job is not to script the play, but to build an unbreakable stage.

In this paradigm, the mechanism—the smart contract, the protocol, or the algorithmic policy—acts as an immutable law of physics. It responds deterministically to a universe of chaotic, unpredictable, and potentially adversarial agent inputs, bounded only by the rigid cryptographic rules of admissibility. This strict epistemological separation ensures that even if we cannot predict the exact trajectory of the system, we can mathematically guarantee its baseline safety, proving that catastrophic failure states remain forever out of reach. Ultimately,

the Generalized Dynamical Systems framework fosters a vital scientific mindset: one defined by profound epistemic humility regarding the unpredictability of human behavior, paired with absolute mathematical rigor regarding the design of system infrastructure, thereby laying the permanent groundwork for the future of cryptoeconomic engineering.

Works cited

1. Generalized Dynamical Systems Part I: Foundations - WU Research, https://research.wu.ac.at/ws/portalfiles/portal/23782375/Zargham_Shorish_GDS_Part_I_Foundations_2022.pdf
2. Logical Analysis of Hybrid Systems: A Complete Answer to a Complexity Challenge¹ - Carnegie Mellon University, <https://www.cmu.edu/traffic21/pdfs/dcfs12-long.pdf>
3. generalized dynamical systems, uploaded:generalized dynamical systems
4. Uncertain Dynamic Systems—A Liapunov Min-Max Approach - ResearchGate, https://www.researchgate.net/publication/3028447_Uncertain_Dynamic_Systems-A_Liapunov_Min-Max_Approach
5. Differential Dynamic Logics - André Platzer, <https://symbolaris.com/pub/diss.pdf>
6. Differential Dynamic Logic for Hybrid Systems - André Platzer, <https://symbolaris.com/pub/freedL.pdf>
7. Formal verification of isothermal chemical reactors - arXiv, <https://arxiv.org/html/2509.01130v1>
8. Architecting the Cyber-Physical Commons | by Jeff Emmett | Commons Stack - Medium, <https://medium.com/commonsstack/architecting-the-cyber-physical-commons-a294d88b5415>
9. Properties of Min-Max Controllers in Uncertain Dynamical Systems, <https://epubs.siam.org/doi/10.1137/0320060>
10. Cryptoeconomics - IDEAS/RePEc, <https://ideas.repec.org/a/zbw/iprjir/235957.html>
11. Tokenomics design for local communities. Interdisciplinary method for co-design - Frontiers, <https://www.frontiersin.org/journals/blockchain/articles/10.3389/fbloc.2026.1758395/full>
12. Token Economies | The Emerald Handbook of Blockchain for Business | Books Gateway, <https://www.emerald.com/books/edited-volume/13510/chapter/84146174/Token-Economies>
13. (PDF) Cryptoeconomic Theory Introduction - ResearchGate, https://www.researchgate.net/publication/391909766_Cryptoeconomic_Theory_Introduction
14. The Commons Stack. Realigning Incentives Towards Public Goods. Case Study, https://www.researchgate.net/publication/347390484_The_Commons_Stack_Realigning_Incentives_Towards_Public_Goods_Case_Study